

5.5 Internet of Things WIFI Control System ESP32

ESP32 IoT WiFi Control System



💡 A Note for Newcomers: Don't panic! This tutorial is written specifically for first-time ESP32 users. Every step is explained in detail — just follow along and you'll succeed. If you run into trouble, jump straight to the Troubleshooting section near the end — almost every issue has an answer there.

📋 What You'll Build

By completing this project, you'll have a smart ESP32 device that you can control from your phone:

- ▶ 📱 Connect your phone to the WiFi hotspot created by the ESP32 and control it via a web page
- ▶ 🕒 View accurate network time and set alarms (buzzer will ring)
- ▶ 🌡️ Real-time temperature and humidity display (both on OLED screen and your phone)
- ▶ 💡 Remotely turn the onboard LED on and off
- ▶ ⚙️ Remotely control a motor/relay (can be extended to fans, lights, etc.)

Simple Analogy: Think of it like using your phone to control your home AC — your phone is the remote, the ESP32 is the AC, and WiFi is the "telephone line" connecting them.

🛠️ Step 1: Prepare Your Tools (Hardware + Software)

Hardware Checklist

Gather the following items (check them off as you go):

#	Component Name	Quantity	Purpose	✅ Done
1	ESP32 Development Board	1 pc	Main controller	☐
2	DHT11 Temperature & Humidity Sensor	1 pc	Measure temp & humidity	☐
3	0.96" OLED Display (128×64, I2C)	1 pc	Show time, temp & humidity	☐
4	Passive Buzzer (3.3V-5V, PWM driven)	1 pc	Alarm ringing (requires PWM square wave)	☐
5	DC Motor	1 set	Remote motor control	☐
6	Push Button (tactile switch)	1 pc	Manually stop alarm when ringing	☐
7	Breadboard	1 pc	For easy wiring	☐
8	Jumper Wires (M-M, M-F)	Several	Connect everything together	☐
9	Micro USB / Type-C Data Cable (NOT charge-only!)	1 pc	Connect PC to flash code	☐

🔧 Step 2: Hardware Wiring



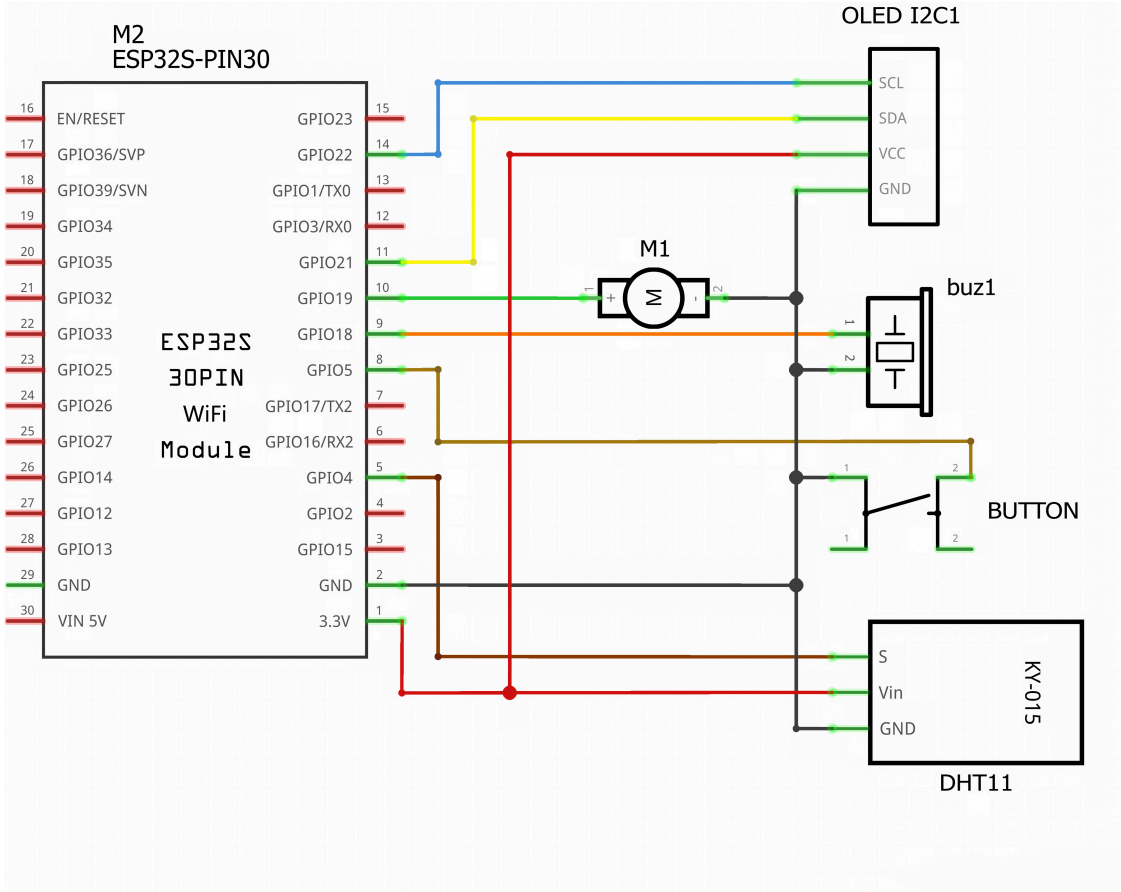
⚠️ CRITICAL SAFETY WARNING: Always **DISCONNECT the ESP32 from USB power** before wiring! Check your connections 3 times before applying power. Wrong wiring can damage your components!

Wiring Table (FOLLOW THIS EXACTLY!)

ESP32 Pin	Connects To	Notes
GPIO4	DHT11 DATA / OUT	Temperature/humidity data pin
GPIO21	OLED SDA	I2C Data Line

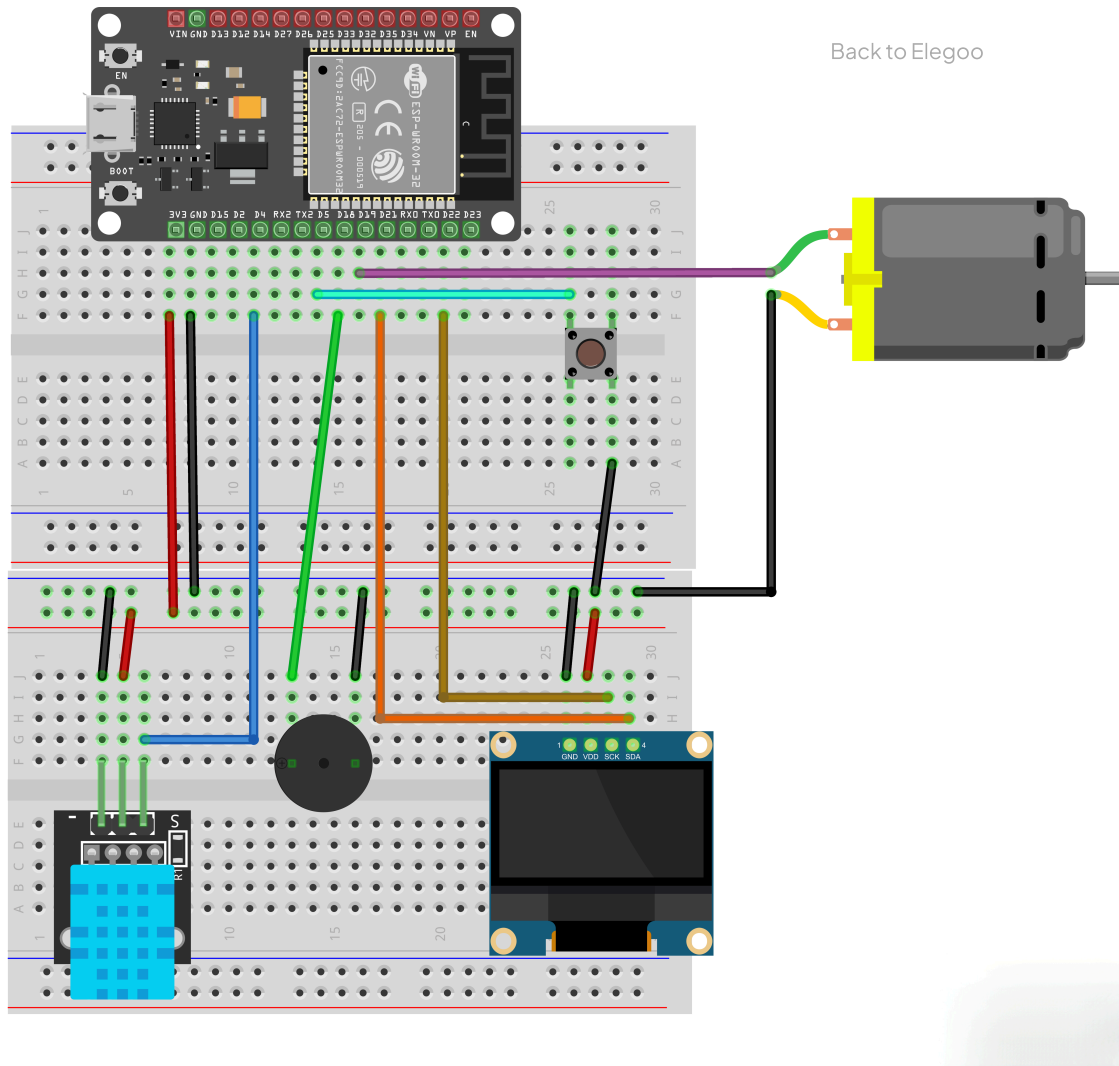
ESP32 Pin	Connects To	Notes
GPIO22	OLED SCL	I2C Clock Line
GPIO18	Passive Buzzer Signal Pin (IN / SIG)	PWM-driven buzzer ringing
GPIO19	Motor Driver Module IN1 / Signal Input	Control motor on/off
GPIO5	Push Button (other terminal)	Manually stop alarm when ringing
GPIO2 (Built-in)	(Already on the board, no wiring needed)	Onboard LED

Connection Schematic



Wiring diagram

[Back to Elegoo](#)



Step 3: Software Setup & Code Flashing (6 Simple Steps)

Step 3.1: Add ESP32 Board Support to Arduino IDE

Step 3.2: Install Required Libraries

Open the libraries folder in the project folder, then load it following the library loading process.

Step 3.3: Open the Project Code

 [DOWNLOAD](#)

1. Copy the entire project folder ([Internet_of_Things_WiFi_Control_System_ESP32](#)) to a location you can easily find (like your Desktop)
2. Double-click the [Internet_of_Things_WiFi_Control_System_ESP32.ino](#) file inside that folder — it will automatically open in Arduino IDE
3. ⚡ Notice the multiple tabs at the top of the IDE: [.ino](#) , [NTPTime.cpp](#) , [OLEDDisplay.cpp](#) , etc. — this is the "multi-file modular" structure. Each file handles one specific job (we'll explain this in detail later).

Step 3.4: Modify WiFi Configuration (THE MOST IMPORTANT STEP!)

 **If you skip this step, WiFi connection WILL fail later!**

1. In the tabs at the top of the IDE window, click to switch to the **WebServer.cpp** file
2. Find this section of code (around line 10):

```
1 //STA mode configuration: Connects to your router to get NTP time
2 const char* sta_ssid = "YourWiFiName"; // ⚡ Change to YOUR WiFi name!
3 const char* sta_password = "YourWiFiPassword"; // ⚡ Change to YOUR WiFi password!
```

3. Replace the text inside the double quotes with **your own home WiFi credentials**:

- ▶ `sta_ssid` : Change to your WiFi name (SSID)
- ▶ `sta_password` : Change to your WiFi password

Example:

```
1 | const char* sta_ssid = "MyHomeNetwork";
2 | const char* sta_password = "mypassword123";
```

4. (Optional) If you want a custom hotspot name and password, modify these too:

```
1 | const char* ap_ssid = "ESP32_AP"; // Name shown in phone WiFi list
2 | const char* ap_password = "12345678"; // Hotspot password — at least 8 chars
```

Step 3.5: Select Board & Flash the Code

Step 3.6: Observe the Boot Process (Serial Monitor)

After successful flashing, let's use the Serial Monitor to see what the ESP32 is doing:

1. Click the  **Serial Monitor** button in the top-right of the IDE (looks like a plug icon), or press **Ctrl+Shift+M**
2. In the bottom-right corner of the Serial Monitor window, set the **baud rate to 115200** (EXTREMELY IMPORTANT! Wrong baud rate = garbage text)
3. Press the **EN Button** (Reset button) on your ESP32 board to restart it
4. Watch the output in the Serial Monitor — compare it against the Expected Output section below

✅ Step 4: Check if Everything Works

4.1 Expected Serial Monitor Output

Case 1: WiFi + NTP Both Connected Successfully (Normal Operation)

```
1 | === Getting NTP time at startup ===
2 | Connecting to WiFi: MyHomeNetwork
3 | ....
4 | ✅ WiFi connected successfully!
5 | NTP sync attempt 1/30: ✅ Success!
6 | Current time: 15:30:45
7 |
8 | 📶 Starting AP hotspot: ESP32_AP
9 | AP IP address: 192.168.4.1
10 | Time: 15:30:46
11 | T = 25.5 deg. C, H = 60.0%
12 | Time: 15:30:47
13 | ...
```

- ▶ See **✅ WiFi connected successfully!** → WiFi connected ✓
- ▶ See **NTP sync attempt 1/30: ✅ Success!** → Time synchronized ✓
- ▶ See **AP IP address: 192.168.4.1** → Hotspot created successfully ✓
- ▶ Time prints once per second → System running normally ✓

Case 2: WiFi Connection Failed (Code still runs!)

```
1 | === Getting NTP time at startup ===
2 | Connecting to WiFi: WrongSSID
3 | .....
4 | ❌ WiFi connection failed, unable to sync NTP time
5 |
6 | 📶 Starting AP hotspot: ESP32_AP
7 | AP IP address: 192.168.4.1
8 | Time: 00:00:01
9 | Time: 00:00:02
10 | ...
```

- ▶ In this case, the internal clock will start from **00:00:00** (like a digital watch that hasn't been set)
- ▶ **However, the hotspot and remote control functions STILL WORK** — only the time/alarm accuracy will be off

4.2 What You Should See on the OLED Screen

After successful OLED initialization, the screen should display:

[Back to Elegoo](#)

```
1 |  
2 | ALARM 08:30      ← Shows here ONLY if alarm is set  
3 | 08:30 45        ← Large Hours:Minutes + Seconds  
4 | T:25.5C  H:60.0% ← Temperature and Humidity  
5 |
```

When sensor fails: If the DHT11 reading fails (wiring issue or sensor fault), the bottom row will show **ERR** instead of values, for example:

```
1 |  
2 | 08:30 45  
3 | T:ERRC  H:ERR%  ← Sensor reading abnormal  
4 |
```

If the screen is blank, go to Troubleshooting Q3 / Q4 below.

4.3 Connect Your Phone and Take Control

1. Open your phone's **WiFi Settings**
2. Find the WiFi network named **ESP32_AP** (or your custom hotspot name) and tap to connect
3. Enter the password: **12345678** (or your custom password)
4. 🗨 After connecting, your phone may warn "This hotspot has no internet access" — this is NORMAL! The ESP32 itself can't go online. If your phone asks "Stay connected?", always choose **Stay Connected / Don't Switch**.
5. Open any web browser on your phone (Chrome, Safari, the default browser — all work fine)
6. Type **192.168.4.1** in the address bar and press Enter / Go
7. 🐛 You should see the control page! Try clicking the buttons:
 - ▶ **LED ON / LED OFF:** The onboard LED on ESP32 lights up / turns off
 - ▶ **Motor ON / Motor OFF:** Motor starts / stops spinning
 - ▶ **Set Alarm:** Enter hour and minute, then click "Set Alarm"
 - ▶ **Turn Off Alarm:** If alarm is ringing, this stops it immediately
 - ▶ **Physical Button Stop:** Besides the web button, you can also press the physical button connected to GPIO5 to stop the alarm directly

🙄 Step 5: Troubleshooting Guide

Q1: Serial Monitor shows garbage text or is completely blank?

A: This is the #1 beginner mistake — almost 100% of the time it's the wrong baud rate.

- ▶ Check the baud rate dropdown in the bottom-right of the Serial Monitor. It **MUST be 115200**.
- ▶ After changing the baud rate, press the EN (Reset) button on the ESP32 to restart it.

Q2: WiFi never connects. Prints many dots (.....) then fails?

A: Check each of these systematically, in order:

1. 🔴 **Verify WiFi Name & Password:**
 - ▶ Open [WebServer.cpp](#) and double-check that `sta_ssid` and `sta_password` are correct
 - ▶ They are case sensitive!
 - ▶ Are there any extra spaces accidentally added?
2. 🔴 **Confirm it's 2.4GHz WiFi:**
 - ▶ ESP32 **does NOT support 5GHz!**
 - ▶ Log into your router admin panel and confirm the 2.4GHz band is enabled
 - ▶ Some routers use the same name for both 2.4GHz and 5GHz — try giving the 2.4GHz network a unique SSID
3. 🔴 **Check Router MAC Address Filtering:**
 - ▶ Does your router have "MAC Address Filtering" or "Access Control" enabled?
 - ▶ If yes, either temporarily disable it or add the ESP32's MAC address to the whitelist
4. 🔴 **Special Characters in Password:**

- If your password contains spaces, non-Latin characters, or unusual symbols, test with a simple alphanumeric password first

5. Signal Strength:

- Is the ESP32 too far from the router? Move it closer and try again.

Q3: OLED shows nothing and Serial says "OLED initialization failed"?

A: OLED failed to initialize. Check each item:

1. Wiring Issues:

- OLED SDA → ESP32 **GPIO21**
- OLED SCL → ESP32 **GPIO22**
- OLED VCC → **3.3V** (Some OLEDs tolerate 5V — try that too)
- OLED GND → ESP32 GND (MUST be common ground!)

2. Wrong I2C Address:

- Most OLED modules use address `0x3C`, but a few use `0x3D`
- Open `OLEDDisplay.cpp`, find the `oledInit()` function
- Try changing `0x3C` to `0x3D` in the `display.begin(...)` call

3. Insufficient Power:

- Some OLED modules are voltage-sensitive. If 3.3V doesn't work, try 5V.

4. Defective Module:

- Tried everything above and still dead? The OLED module itself might be faulty. Try another one.

Q4: OLED doesn't report "initialization failed" but the screen is blank or very dim?

A: This is almost always a contrast/brightness issue. In `OLEDDisplay.cpp`, in the `oledInit()` function, add these two lines AFTER `display.begin(...)`:

```
1 | display.ssd1306_command(SSD1306_SETCONTRAST);
2 | display.ssd1306_command(0xCF); // 0x00 = dimmest, 0xFF = brightest. Adjust to taste.
```

Q5: DHT11 readings are always 0.0?

A: Check each item:

1. Wiring Issues:

- DHT11 DATA → ESP32 **GPIO4**
- DHT11 VCC → 3.3V (ESP32 is 3.3V logic, do NOT use 5V)
- DHT11 GND → ESP32 GND (MUST be common ground!)

2. 4.7kΩ Pull-Up Resistor:

-  A **4.7kΩ pull-up resistor** MUST be connected between DHT11 DATA and VCC! This is required by the DHT11's open-drain output
- Some DHT11 modules have this resistor built-in (check the back of the module for an "S" mark or resistor)
- Without the pull-up resistor, the DATA pin level is undefined and readings will fail

3. Pin Order:

-  DHT11 modules come in TWO different pinouts! Always check the silk screen on the BACK of your module:
 - Variant A: VCC → DATA → GND (left to right)
 - Variant B: DATA → VCC → GND (left to right)
- Wrong pin order = either no readings OR permanent sensor damage!

4. Warm-Up Time:

- DHT11 needs 1~2 seconds to stabilize after power-on. The code already includes a 2-second wait
- DHT11 minimum sampling interval is 2 seconds — do not read too frequently

5. Dead Sensor:

- Try a different DHT11 module

Q6: Phone can't find the "ESP32_AP" hotspot?

A:

1. Was the code actually flashed successfully?

- Check that the IDE showed "Done uploading"
- Open the Serial Monitor at 115200 baud, press EN to reset, and look for output

2. Press the EN Button:

- After flashing, press the EN (Reset) button once to ensure the code is running

3. Password Length Check:

- AP password must be at least 8 characters. Fewer than 8 = hotspot creation failure.

Q7: Phone connects to hotspot but browser can't open 192.168.4.1?

A:

1. Did your phone auto-switch WiFi?

- Many Android phones detect that the hotspot has "no internet" and automatically switch back to a WiFi network that does have internet
- Fix: In phone WiFi settings, **DISABLE "Smart Switch", "WiFi+", "Auto-switch", or similar options**
- Or explicitly select the option to **"Stay Connected" / "Don't Switch"**

2. Browser using Proxy / VPN?

- Turn off any proxy, VPN, or ad-blocking services on your phone

3. Try Incognito / Private Browsing Mode:

- Open an **Incognito / Private Browsing** window in the browser and try entering [192.168.4.1](#) again

4. Manually Set Static IP on Phone:

- In phone WiFi settings, long-press [ESP32_AP](#) → Modify Network → Static IP settings:
 - IP: [192.168.4.2](#)
 - Gateway: [192.168.4.1](#)
 - Subnet Mask: [255.255.255.0](#)

Q8: Web page opens but clicking buttons does nothing?

A:

1. Check for output in Serial Monitor:

- When you click a button, the Serial Monitor should print something like [Request: GET /H HTTP/1.1](#)
- If NO output → Browser request never reached the ESP32 (go back and fix Q7)
- If OUTPUT shows up but LED doesn't light → Possible code logic issue

2. Refresh the page and try again:

3. Restart the ESP32 (press EN button):

Q9: Alarm doesn't ring at the set time?

A:

1. Wiring Issues:

- Buzzer Signal Pin → ESP32 **GPIO18**
- Buzzer VCC → 3.3V or 5V
- Buzzer GND → ESP32 GND (MUST be common ground!)

2. Buzzer Type:

-  **This project uses a PASSIVE buzzer** (PWM driven):
 - **Passive Buzzer:** Requires a PWM square wave signal (code uses [tone\(\)](#) to output 1000Hz square wave) — DC voltage alone won't make it sound →  This project is designed for these
 - **Active Buzzer:** Rings immediately when you apply DC voltage →  Won't work with this code as-is
- How to test: Connect buzzer VCC to 3.3V and GND directly to GND — if it rings, it's active (won't work with this code); if silent, it's passive (correct for this project)

3. Is the Clock Correct?

- Compare the time shown on the OLED against the actual current time
- If time starts from 00:00, the NTP sync failed (go fix Q2 first)

4. Was the Alarm Actually Set?

[Back to Elegoo](#)

- ▶ After setting the alarm on the web page, does **ALARM XX:XX** appear at the top of the OLED?
- ▶ Did the Serial Monitor print  **Alarm set: XX:XX** ?

5. Alarm triggers at the 0th SECOND of the target minute:

- ▶ Example: If you set alarm 08:30, it will trigger ONLY at exactly 08:30:00 (the very first second of that minute) — it will NOT ring continuously through the entire 08:30 minute
- ▶ For quick testing, set the alarm to "current time + 1 minute"

6. Use Physical Button to Stop Alarm:

- ▶ Besides the "Turn Off Alarm" web button, you can also press the physical button on GPIO5 to stop the alarm ringing directly

Q10: Motor doesn't spin?

A:

1. Wiring Issues:

- ▶ Motor Driver Module IN1 → ESP32 **GPIO19**
- ▶ Motor Driver Module GND → ESP32 GND (MUST be common ground!)
- ▶ Motor's two wires → Driver output terminals (OUT1 / OUT2)

2. Insufficient starting current?

- ▶  **The ESP32 can only drive the motor temporarily!** Be sure to use a driver module such as L293D for long-term operation. If the motor fails to rotate when pressing the button, gently turn the output shaft with your finger to assist startup. Please turn off the motor promptly once testing is completed.

3. Check Serial Monitor Output:

- ▶ When you click the button, the Serial Monitor should print **Motor state updated: ON**
- ▶ If output appears but motor doesn't move → Problem is in motor wiring / driver / power supply
- ▶ If NO output appears → Go back and fix Q8 first

 Quick Troubleshooting Lookup Table (Fast Diagnosis)

Symptom	Most Likely Cause	Fastest Fix
Serial garbage / blank	Wrong baud rate	Set to 115200, press EN to reset
WiFi won't connect (many dots)	Wrong password / 5GHz WiFi	Verify credentials, use 2.4GHz
OLED blank + error message	Wrong wiring / wrong I2C address	Check I2C wiring, try address 0x3D
OLED blank + NO error	Contrast too low	Add contrast adjustment code
OLED shows T:ERR / H:ERR	DHT11 wiring issue / sensor fault	Check GPIO4 wiring and 4.7kΩ pull-up resistor
DHT11 always 0.0	Wrong wiring / reversed pin order	Check GPIO4, verify pinout on silk screen
Can't find hotspot	Code not flashed properly	Re-flash code, press EN to reset
Can't open web page	Phone auto-switched WiFi	Disable auto-switch, try incognito mode
Buttons unresponsive	Browser cache	Refresh page / restart ESP32
Alarm silent	Wrong wiring / wrong buzzer type	Check GPIO18, use passive buzzer (PWM driven)
Motor won't move	No driver / no external power	Add motor driver, connect motor power supply
Physical button can't stop alarm	Button wiring / no pull-up	Check GPIO5 wiring, button's other terminal to GND

 Step 6: Background Knowledge (Read This After Getting it Working!)



 Congratulations if you've already gotten the project working! This section is for those who want to understand the "why" behind the code. First-timers can safely skip this and just enjoy the feeling of success.

6.1 What is an IoT Control System?

[Back to Elegoo](#)

IoT (Internet of Things) is all about connecting devices together through a network so they can communicate and be controlled remotely.

This project is a classic three-layer IoT application:

- **Perception Layer (Eyes & Ears):** DHT11 sensor feels the environment's temperature and humidity
- **Transport Layer (Nerves):** WiFi sends data out and receives commands in
- **Application Layer (Brain):** You, using a web page on your phone, view data and send control commands

6.2 Project Architecture: Why Multi-File?

The project code is split into **9 files**, each responsible for one specific thing. This is called **multi-file modular programming**.

File Name	What It Does	Real-World Analogy
Internet_of_Things_WIFI_Control_System_ESP32.ino	Main program, the boss	Company CEO
NTPTime.h / .cpp	Connects WiFi, fetches network time	Admin Dept — keeps the clock accurate
DHTSensor.h / .cpp	Reads DHT11 temp & humidity	Environmental monitor
OLEDDisplay.h / .cpp	Controls OLED screen display	Receptionist — shows info to visitors
WebServer.h / .cpp	Creates hotspot, handles web requests	Customer Service Dept

Module Relationship Diagram:

- ```
1 | Main Program (.ino) — calls → NTPTime Module (get time)
2 | ↳ DHTSensor Module (read temp & humidity)
3 | ↳ OLEDDisplay Module (display info)
4 | ↳ WebServer Module (create hotspot + web page)
```

Why Use Modular Programming?

| Advantage     | Explanation                                                                                |
|---------------|--------------------------------------------------------------------------------------------|
| Easy to Debug | If temp/humidity is wrong, just look in DHTSensor.cpp — no need to hunt through 1000 lines |
| Easy to Reuse | Need OLED in your next project? Just copy OLEDDisplay.h/.cpp straight over!                |
| Clear Logic   | Each file does one thing — fewer chances of bugs                                           |

6.3 STA Mode vs AP Mode

Boot-Up Sequence Used in This Project:

1. First use **STA Mode** to connect to your home router → Fetch accurate time from an NTP server
2. After obtaining the time, disconnect from STA mode
3. Switch to **AP Mode** to create a hotspot → Your phone can connect and take control

Why switch modes? While ESP32 technically supports simultaneous STA+AP (hybrid mode), in practice it causes many resource conflicts and poor stability. Sequential switching ("STA for time → disconnect → start AP") gives far higher reliability.

6.4 Why Use `millis()` Instead of `delay()` ?

This is a very important concept in embedded programming — **non-blocking programming**.

❌ **The Problem with `delay(1000)`** : It "freezes" the entire program for 1 full second. During that frozen time, it can't ring alarms, can't handle web requests, can't read buttons.

✅ **The Beauty of `millis()`** : `millis()` returns "how many milliseconds have elapsed since boot". We use this to check if "enough time has passed" — if not, we move on and do other things.

- ```
1 | // Correct approach: Non-blocking
2 | if (millis() - lastTime >= 1000){
3 |   lastTime = millis();
4 |   // 1 second is up — execute the timed task
5 |
6 | }
```

```
}  
// Code below here keeps running every single loop — never gets stuck!
```

[Back to Elegoo](#)

🔗 **About the `static` keyword:** A `static` variable declared inside a function is NOT destroyed when the function exits — the next time the function is called, it still remembers its old value. Think of it as "memory" inside a function — perfect for remembering when we last did something.

🔗 **`tone()` / `noTone()` — PWM Drive for Passive Buzzer:** This project uses a passive buzzer which requires a PWM square wave to produce sound. `tone(pin, 1000)` generates a 1000Hz square wave on the pin, and `noTone(pin)` stops it. Compared to manually flipping pin levels with `millis()`, `tone()` is implemented by hardware timers and doesn't consume any CPU time — much more efficient.

🔗 **Button Detection — Falling Edge Trigger:** The physical button uses "falling edge detection" — it only triggers once at the exact moment the button transitions from released (HIGH) to pressed (LOW), rather than continuously triggering while held down. This is implemented using a `static` variable to remember the previous button state.

🔍 Step 7: Detailed Code Walkthrough (Advanced Learning)

7.1 Main Program File: .ino

7.1.1 Headers and Pin Definitions

```
1  #include <WiFi.h>      // ESP32 WiFi library (NOT WiFIS3.h!)  
2  #include <Wire.h>      // I2C communication (for OLED)  
3  #include "OLEDDisplay.h" // OLED display module  
4  #include "WebServer.h"  // Web server module  
5  #include "NTPTime.h"    // NTP time module  
6  #include "DHTSensor.h"  // Temp/humidity sensor module  
7  
8  // ===== Buzzer (passive, PWM driven) =====  
9  #define BUZZER_PIN 18   // Buzzer → GPIO18  
10 #define BUZZER_FREQ 1000 // 1000Hz PWM frequency (passive buzzer needs square wave)  
11  
12 // ===== Buzzer Stop Button =====  
13 #define BUZZER_BTN_PIN 5 // Button → GPIO5  
14  
15 // ===== Motor Control =====  
16 #define MOTOR_PIN 19     // Motor → GPIO19  
17  
18 // ===== LED =====  
19 #define LED_PIN 2        // Onboard LED → GPIO2  
20  
21 bool motorState = false; // Motor on/off state variable  
22  
23 // Global variables  
24 unsigned long currentTime = 0; // Current timestamp (seconds)  
25 unsigned long lastUpdateTime = 0; // Last update time (milliseconds)  
26  
27 // Alarm variables  
28 int alarmHour = 0;  
29 int alarmMin = 0;  
30 bool alarmOn = false; // Whether alarm is enabled  
31 bool alarmRing = false; // Whether alarm is currently ringing  
32  
33 // Temperature & Humidity variables  
34 float temperature = 0.0;  
35 float humidity = 0.0;
```

7.1.2 `setup()` : Initialization Function (Runs ONCE at boot)

```
1  void setup(){  
2    Serial.begin(115200); // Initialize serial at 115200 baud  
3    delay(1000);  
4  
5    pinMode(LED_PIN, OUTPUT); // Set LED pin as output  
6    pinMode(MOTOR_PIN, OUTPUT); // Set motor pin as output  
7    digitalWrite(LED_PIN, LOW); // Initial state: LED OFF  
8    digitalWrite(MOTOR_PIN, LOW); // Initial state: motor OFF  
9  
10   // Buzzer stop button (active low with internal pull-up)  
11   pinMode(BUZZER_BTN_PIN, INPUT_PULLUP);
```

```

12
13 // Wait for DHT11 sensor to stabilize
14 Serial.print("[DHT] Sensor type: DHT11, Pin: GPIO");
15 Serial.println(DHT_SENSOR_PIN);
16 Serial.println("[DHT] Waiting for sensor stabilization...");
17 delay(2000);
18
19 Wire.begin(21, 22); // I2C init: SDA=GPIO21, SCL=GPIO22
20 oledInit(); // Initialize OLED display
21
22 // ===== Boot-up Sequence =====
23 getNetTime(currentUnixTime); // Step 1: Connect WiFi, get NTP time
24 startAPMode(); // Step 2: Switch to AP mode, create hotspot
25 server.begin(); // Step 3: Start Web server
26
27 lastUpdateTime = millis(); // Step 4: Record timekeeping reference point
28 }

```

7.1.3 loop() : Main Loop (Runs REPEATEDLY forever)

The main loop contains **4 core tasks**, each implemented in a non-blocking way:

Core 1: Poll DHT sensor + per-second timekeeping + display update

```

1 // Poll DHT sensor every loop iteration (module internally handles 3s non-blocking sampling)
2 measureEnvironment(&temperature, &humidity);
3
4 if (millis() - lastUpdateTime >= 1000) {
5   lastUpdateTime = millis();
6   currentUnixTime++; // Increment timestamp +1 second
7
8   // Split timestamp into hours, minutes, seconds (using modulo arithmetic)
9   int h = (currentUnixTime % 86400) / 3600; // 86400 = seconds per day
10  int m = (currentUnixTime % 3600) / 60; // 3600 = seconds per hour
11  int s = currentUnixTime % 60;
12
13  // Print time (zero-padded for neat display)
14  Serial.print("Time: ");
15  if (h < 10) Serial.print("0"); Serial.print(h);
16  Serial.print(":");
17  if (m < 10) Serial.print("0"); Serial.print(m);
18  Serial.print(":");
19  if (s < 10) Serial.print("0"); Serial.println(s);
20
21  // Update OLED display
22  updateOLED(h, m, s, alarmHour, alarmMin, alarmOn, temperature, humidity);
23
24  // Alarm trigger check (fires ONLY at exactly the 0th second of target minute)
25  if (alarmOn && h == alarmHour && m == alarmMin && s == 0) {
26    alarmRing = true;
27  }
28 }

```

Core 2: Handle web requests (checked EVERY loop iteration)

```

1 | handleWebRequest(currentUnixTime, alarmHour, alarmMin, alarmOn, alarmRing, BUZZER_PIN, temperature, humidity, motorState);

```

Core 3: Alarm ringing (PWM-driven passive buzzer + button stop)

```

1 // Buzzer button: press to stop alarm
2 static bool lastBtnState = HIGH;
3 bool btnState = digitalRead(BUZZER_BTN_PIN);
4 if (btnState == LOW && lastBtnState == HIGH) {
5   delay(20); // simple debounce
6   btnState = digitalRead(BUZZER_BTN_PIN);
7   if (btnState == LOW) {
8     if (alarmRing) {
9       alarmRing = false;
10      alarmOn = false;
11      noTone(BUZZER_PIN);
12      Serial.println("Button pressed: alarm stopped");
13    }

```

```

14 | }
15 | }
16 | lastBtnState = btnState;
17 |
18 | // Alarm ringing (PWM square wave drives passive buzzer)
19 | if (alarmRing) {
20 |   tone(BUZZER_PIN, BUZZER_FREQ); // Output 1000Hz PWM square wave
21 | } else {
22 |   noTone(BUZZER_PIN); // Stop PWM output
23 | }

```

[Back to Elegoo](#)

Core 4: Motor state change detection (only write to pin when state actually changes)

```

1 | static bool lastMotorState = false;
2 | if (motorState != lastMotorState) {
3 |   lastMotorState = motorState;
4 |   digitalWrite(MOTOR_PIN, motorState ? HIGH : LOW);
5 |   Serial.print("Motor state updated: ");
6 |   Serial.println(motorState ? "ON" : "OFF");
7 | }

```

7.2 Key Function Quick Reference

WiFi Functions (ESP32-Specific)

Function	Purpose
WiFi.mode(WIFI_STA)	Switch to STA mode (preparing to connect to router)
WiFi.begin(ssid, pass)	Begin connecting to specified WiFi
WiFi.status()	Check connection status (WL_CONNECTED = success)
WiFi.disconnect(true)	Disconnect current WiFi connection
WiFi.mode(WIFI_OFF)	Turn off WiFi module completely
WiFi.softAP(ssid, pass)	Create AP hotspot (ESP32 specific! UNO R4 uses beginAP())
WiFi.softAPIP()	Get ESP32's own IP in AP mode (default 192.168.4.1)

Arduino Core Functions

Function	Purpose
tone(pin, frequency)	Output PWM square wave on specified pin. frequency sets the wave frequency in Hz, e.g. tone(18, 1000) outputs 1000Hz to drive passive buzzer
noTone(pin)	Stop PWM square wave output — turns off tone() output, stops passive buzzer completely
millis()	Returns milliseconds since boot — used for non-blocking timing: millis() - lastTime >= 1000 checks if 1 second has passed

HTTP Request Routing Table

When you click buttons on the web page, your browser sends these HTTP requests. The ESP32 parses them and takes the corresponding action:

Request Path	Function	ESP32 Action
GET /H	Turn LED ON	digitalWrite(LED_BUILTIN, HIGH)
GET /L	Turn LED OFF	digitalWrite(LED_BUILTIN, LOW)
GET /MOTOR_ON	Turn Motor ON	motorState = true
GET /MOTOR_OFF	Turn Motor OFF	motorState = false

Request Path	Function	ESP32 Action
GET /ALARM?h=08&m=30	Set Alarm	Parse h and m, set alarm time, <code>alarmRing = false</code>
GET /ALARM_OFF	Turn Off Alarm	<code>alarmRing = false; alarmOn = false; alarmHour=0; alarmMin=0; noTone(BUZZER_PIN);</code>

Step 8: Common Modification Guide

Ready to customize? Start here!

8.1 Modify WiFi & Hotspot Settings

Open [WebServer.cpp](#) :

```
1 | const char* sta_ssid = "YourHomeWiFi";
2 | const char* sta_password = "YourHomePassword";
3 | const char* ap_ssid = "MyESP32"; // Custom hotspot name
4 | const char* ap_password = "MySecurePass88"; // Hotspot password (min 8 chars)
```

8.2 Modify Timezone & NTP Server

Open [NTPTIME.cpp](#) :

```
1 | const char* ntpServer = "pool.ntp.org"; // Global NTP pool
2 | //const char* ntpServer = "time.google.com"; // Google time server
3 | //const char* ntpServer = "ntp.jst.mfeed.ad.jp"; // Japan NTP server
4 |
5 | const long gmtOffset_sec = 0 * 3600; // UTC (London)
6 | //const long gmtOffset_sec = 9 * 3600; // Tokyo (UTC+9)
7 | //const long gmtOffset_sec = -5 * 3600; // New York (UTC-5)
```

8.3 Modify Pin Assignments

What to Change	Where to Change
DHT11 data pin	DHTSensor.h → <code>#define DHT_SENSOR_PIN</code>
Buzzer pin	Main .ino file → <code>#define BUZZER_PIN</code>
Motor pin	Main .ino file → <code>#define MOTOR_PIN</code>
LED pin	Main .ino file → <code>#define LED_PIN</code>
I2C pins	Main .ino file → <code>setup()</code> → <code>Wire.begin(SDA, SCL)</code>

8.4 Add a Feature: Auto Fan When Temperature Exceeds 30°C

Add this block at the END of the main program's `loop()` function (AFTER Core 4):

```
1 | //===== Core 5: Auto temperature fan control (check every 5 seconds) =====
2 | static unsigned long lastAutoCtrlTime = 0;
3 | if(millis() - lastAutoCtrlTime >= 5000) {
4 |   lastAutoCtrlTime = millis();
5 |   if(temperature > 30.0 && !motorState){
6 |     motorState = true;
7 |     Serial.println("🌡 Temperature > 30°C, auto fan ON");
8 |   } else if(temperature <= 28.0 && motorState){
9 |     motorState = false;
10 |    Serial.println("🌡 Temperature <= 28°C, auto fan OFF");
11 |   }
12 | }
```

Step 9: ESP32 Critical Usage Notes

9.1 These Pins Should NEVER Be Used!

Some ESP32 pins are connected to onboard SPI Flash or have special boot functions — using them as regular GPIO will cause crashes:

Pins	Why NOT to Use	Recommendation
GPIO6 ~ GPIO11	Connected to onboard SPI Flash	 ABSOLUTELY DO NOT USE! Will crash / brick your board
GPIO0	Boot/download pin — must be HIGH at power-on	Avoid using as output
GPIO2	Must be LOW during firmware download	OK for LED use in this project, caution with other devices
GPIO12	MTDI pin — affects SPI voltage at boot	Best to avoid
GPIO15	MTDO pin — must be LOW at power-on	Best to avoid

 **List of Safe, Recommended GPIOs:**
4, 5, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33
(Pins 34, 35, 36, 39 are **input-only** — they cannot output)

 9.2 GPIO Logic Level is 3.3V, NOT 5V!

- ▶ ALL ESP32 GPIO pins output **3.3V** for HIGH level
- ▶ Feeding 5V into a GPIO input may damage the ESP32 (even though some claim "5V tolerance", it's not officially recommended)
- ▶ Driving 5V devices requires a "level shifter" or driver module in between ,or use vin pin
- ▶ When driving LEDs, always add a 220Ω to 1kΩ current-limiting resistor in series

[← NTP Time](#)

[Dual Board→](#)